

XIS

TECHNICAL HIRING ASSESSMENT

AI & Computer Vision Department

Company	XIS
Department	AI / Computer Vision
Document Type	Technical Evaluation Framework
Domain	Image Segmentation & Metric Measurement
Difficulty	Intermediate to Advanced
Version	1.0

1 Purpose

This document defines the structured technical assessment framework for evaluating AI/Computer Vision engineering candidates at XIS. The task centres on building a fully working pipeline that:

- Segments a custom object of the candidate's choosing
- Applies camera calibration to remove lens distortion from imagery
- Trains a deep-learning model on self-collected and self-labelled data
- Computes real-world metric measurements (width & height in mm) from pixel data using a calibrated reference object

The assessment evaluates not only implementation correctness but also:

- Understanding of camera geometry and calibration theory
- Model selection rationale and training discipline
- Measurement accuracy and scientific rigour
- Code quality, version control, and documentation

2 Mandatory Requirements (All Steps)

The following requirements are non-negotiable for every submission regardless of Step.

2.1 Version Control - Git Discipline

- Maintain a clean, well-structured Git repository
- Use meaningful, descriptive commit messages
- Show incremental progress - do not submit a single-dump commit
- Use feature branches where appropriate

2.2 Large File Hosting - Google Drive / OneDrive Only

Large image and dataset files must NOT be uploaded to GitHub. All large files - including but not limited to calibration images, raw dataset images, labelled dataset exports, and trained model weights - must be uploaded to **Google Drive or OneDrive**. Only a **shareable link** to those files should be included in the GitHub repository (e.g. in the relevant README.md or folder-level README).

Refusing to follow this instruction will seriously affect the assessment outcome, regardless of the technical quality of the submission.

Guidelines:

- Keep the GitHub repository lightweight - code, configs, documentation, and small sample files only
- Ensure Drive/OneDrive links are set to "Anyone with the link can view" so reviewers can access them without requesting permission
- Clearly label each link with what it contains (e.g. "Calibration Images", "Full Dataset", "Trained Model Weights")

- Broken, restricted, or expired links will be treated as missing deliverables

2.3 Technical Documentation (Mandatory - Rejection on Omission)

Every candidate must submit professional documentation alongside the code. **Failure to provide documentation will result in automatic rejection.**

Minimum documentation structure:

- **Project Overview** - what the system does and its key capabilities
- **Setup & Installation Guide** - steps, environment, Docker if applicable
- **System / Pipeline Architecture** - end-to-end data flow diagram and explanation
- **Camera Calibration Report** - method, calibration images, intrinsic matrix, distortion coefficients
- **Dataset Description** - object chosen, collection strategy, labelling tool, class distribution
- **Model Training Report** - architecture, hyperparameters, metrics (mAP, IoU, loss curves)
- **Measurement Methodology** - pixel-to-mm conversion derivation, reference object, accuracy analysis
- **API / Module Documentation** - endpoints or function signatures with sample input/output
- **Design Decisions** - approach rationale and trade-offs
- **Assumptions & Limitations**

Documentation will be evaluated on:

- Clarity and logical structure
- Technical depth and correctness
- Completeness and professional presentation

This assessment also evaluates your ability to follow instructions. A significant part of your score depends on how precisely you follow the requirements laid out in this document - file structure, naming conventions, required sections, submission format, and the large-file hosting rule in Section 2.2. This is intentional: at XIS, the ability to read, understand, and accurately execute written instructions and documentation is treated as a core engineering skill, not a formality. Technically strong submissions that disregard stated instructions will be scored lower than submissions that faithfully follow them.

3 Task Overview

Build an end-to-end computer vision pipeline that:

1. Selects a physical object of your choice as the measurement target
2. Performs intrinsic camera calibration to undistort all captured images
3. Collects and labels a custom dataset of that object
4. Trains an image segmentation model

5. Uses calibrated pixel data to measure the real-world width and height of the object in millimetres

The pipeline reflects a real-world industrial measurement workflow. Each stage builds on the previous, and the final system must be demonstrably accurate and reproducible.

4 Step Breakdown

4.1 Step 1 - Foundational: Camera Calibration & Data Collection

Objective: Demonstrate a working camera calibration pipeline and a clean labelled dataset.

Phase	Required Tasks
Object Selection	Choose a physical object. Document its approximate real-world dimensions. Justify your choice (availability, geometry, labelling ease).
Camera Calibration	Capture 20+ calibration images of a checkerboard/ChArUco board from varied angles. Run intrinsic calibration.
Data Collection	Collect 70+ images of the chosen object using the calibrated camera.
Data Labelling	Label images using a tool (CVAT, Roboflow, etc.) and export images.
Documentation	Calibration report (images, parameters, error), dataset card (count, splits, class distribution), setup guide.

Step 1 Expected Outcomes

- Calibrated camera with documented parameters
- Clean labelled dataset ready for training
- All images undistorted and verified

4.2 Step 2 - Intermediate: Model Training & Segmentation

Objective: Train a working segmentation model and evaluate its performance.

Phase	Required Tasks
Model Selection	Select at least one architecture model of your choice other than Roboflow's models and Ultralytics' YOLO models. Justify your choice.
Training Setup	Split dataset: 70% train / 20% val / 10% test. Configure hyperparameters: epochs, batch size, learning rate, augmentation strategy.
Training & Evaluation	Train model and log: loss curves (train/val), mAP@0.5 and mAP@0.5:0.95, IoU, precision, recall, F1. Visualise predictions on a held-out test set.
Inference Pipeline	Build an inference script that accepts input, runs undistortion, runs model inference, and outputs annotated results with detected mask.
Documentation	Model card (architecture, training config, metrics), training log, inference usage guide.

Step 2 Expected Outcomes

- Trained model with documented performance metrics
- Reproducible training pipeline (config file or notebook)
- Working inference on new images from calibrated camera
- Structured documentation with training report

4.3 Step 3 - Advanced: Pixel-to-MM Measurement

Objective: Compute accurate real-world metric measurements from calibrated pixel data.

Phase	Required Tasks
Pixel-to-MM Conversion	Derive the conversion ratio. Apply this ratio to the object's segmentation mask. Calculate: Width (mm) & Height (mm).
Calibration Dependency	All images used for measurement MUST be undistorted using the intrinsic calibration from Step 1. Document why raw (distorted) images produce incorrect measurements.
Accuracy Validation	Measure 10+ instances of the object with a physical ruler/caliper. Compare ground-truth vs. system output. Report mean absolute error (MAE) and mean percentage error (MPE).
End-to-End Demo	Provide a script or notebook that takes a single new image as input and outputs: segmentation mask overlay, width (mm), height (mm), confidence score.
Documentation	Measurement methodology write-up, accuracy report with error table, limitations discussion, end-to-end usage guide.

Step 3 - Expected Outcomes

- Accurate pixel-to-mm measurement with documented error analysis
- End-to-end pipeline from raw image to metric output
- Comprehensive accuracy validation against physical measurements
- Strong documentation explaining the calibration dependency

5 Technical Guidance & Allowed Tools

5.1 Camera Calibration

- Library: OpenCV.
- Calibration target: checkerboard (minimum 7×9 inner corners) or ChArUco board
- Minimum calibration images: 20, covering varied angles, distances, and positions
- Report reprojection error - values below 0.5 px are acceptable; below 0.3 px is excellent
- Intrinsic calibration removes lens distortion (radial and tangential) - this is mandatory before any measurement

5.2 Measurement Pipeline & Conceptual Flow

- Capture image with calibrated camera
- Apply `cv2.undistort()` using stored intrinsic parameters

- Detect reference object and compute `pixels_per_mm` ratio
- Run model inference to segment target object
- Extract pixel dimensions from bounding box or mask contour
- Convert pixel dimensions to millimetres using the ratio
- Output annotated image with metric labels

6 Submission Requirements

6.1 Repository Structure

The submitted Git repository must follow this structure (or equivalent):

```
project-root/
  calibration/      # Calibration images and scripts
  dataset/          # Raw images and labels (train/val/test splits)
  models/           # Training configs and saved weights
  inference/        # Inference scripts and demo outputs
  measurement/      # Pixel-to-mm pipeline and accuracy report
  docs/             # All documentation files
  requirements.txt  # Python dependencies
  README.md         # Project overview and quick-start
```

6.2 Mandatory Documentation Files

- `README.md` - project overview, quick-start, and repository guide
- `CALIBRATION_REPORT.md` - method, parameters, reprojection error
- `DATASET_CARD.md` - object, collection method, labelling tool, statistics
- `TRAINING_REPORT.md` - architecture, hyperparameters, metrics, loss curves
- `MEASUREMENT_REPORT.md` - methodology, accuracy table, error analysis
- `SETUP.md` - full installation, environment, and run instructions

7 Final Evaluation Criteria

Criterion	What is Evaluated
Camera Calibration	Calibration method, reprojection error, undistortion applied correctly
Dataset Quality	Image diversity, labelling accuracy, class balance, dataset size
Model Performance	mAP, IoU, precision/recall, segmentation quality
Measurement Accuracy	Pixel-to-mm conversion correctness, error analysis, reference usage
Code Quality	Readability, modularity, error handling, reproducibility
Documentation	Completeness, technical depth, clarity, professional presentation
Git Discipline	Commit history, message quality, incremental progress
Instruction Adherence	How precisely the candidate followed the stated requirements, formatting, and submission rules in this document

Criterion	What is Evaluated
File Hosting Compliance	Large files correctly hosted on Google Drive/OneDrive with working shareable links; GitHub repository kept lightweight

Documentation quality is a critical factor. Submissions lacking documentation will be rejected without technical review. Similarly, failure to follow the large-file hosting rule (Section 2.2) will seriously affect the assessment outcome.

8 Final Note

This assessment reflects real engineering expectations at XIS. Candidates are expected to demonstrate a thorough understanding of computer vision fundamentals - not just the ability to run a pre-trained model, but the ability to build a calibrated, validated, and well-documented measurement system from the ground up.